Architecture: MCP Relay

Status: Draft v0.1
Last Updated: 2026-02-12
Protocol Version: 2025-11-25

Contents

1. Overview
2. System Context
3. Architecture Approach
4. Project Structure
5. Visual References
6. Supporting Documentation

---

1. Overview

A centralised MCP server that provides shared AI tooling to teams. One server, one set of credentials, a curated tool catalogue — any MCP-compatible client connects with zero individual setup.

Implements the Model Context Protocol (MCP). While primarily validated with Claude-based clients (Claude Code, Cowork, Claude Desktop), the architecture is client-agnostic.

User Journey

```
Team Member                    MCP Server                    External Services
    │                              │                                 │
    ├── Connects MCP client ──────►│                                 │
    │  (Code/Desktop/VS Code/      │                                 │
    │   Cowork/API agent)          │                                 │
    │                              │                                 │
    ├── Invokes tool ─────────────►│                                 │
    │  (e.g. "search Jira")        │                                 │
    │                              ├── Validates input (Zod) ────────►│
    │                              ├── Authenticates (server creds) ─►│ Jira / S3
    │                              ├── Executes API call ────────────►│
    │                              ◄── Receives response ────────────┤
    │                              ├── Validates response (Zod) ─────►│
    │                              ├── Transforms to MCP content ────►│
    │◄── Formatted result ─────────┤                                 │
```

POC Scope

In scope: Jira tools (get, create, search, update), Brand Guidelines (S3 config fetch), shared auth (env vars), stdio transport, Streamable HTTP transport, per-user auth (SSO via OAuth 2.1), production hosting (internal team), Zod validation, error handling, rate limiting hooks, structured logging

Out of scope: Azure DevOps, SonarQube, CI/CD tools, Nanobanana/Gemini, plugin system

Target Users

User Type	Client	Use Case
Developers	Claude Code, VS Code	Jira queries, brand guideline lookups
Delivery teams	Claude Code, Cowork	Jira issue management, project config access
Strategists	Cowork (TBC)	TBD; likely project planning and solution design tooling
Non-technical users	Cowork	Natural language access to Jira and brand guidelines
CI/CD agents (Phase 2+)	API consumers	Headless tool invocation

2. System Context

```
MCP Clients                      MCP Server                    External Services

┌──────────────────┐             ┌──────────────────────┐      ┌──────────────────────┐
│ Claude Code      │──stdio──►   │                      │      │                      │
│ Claude Desktop   │──stdio──►   │  mcp-relay           │──────►  Jira REST API v3     │
│ VS Code          │──stdio──►   │                      │      │  (Cloud)             │
│ Cowork           │──HTTP───►   │  ┌─────────────────┐ │      └──────────────────────┘
│ Other MCP        │──HTTP───►   │  │ Tool Registry   │ │
│   clients        │             │  │  ├─ Jira        │ │      ┌──────────────────────┐
└──────────────────┘             │  │  ├─ Brand       │ │──────►  AWS S3               │
                                 │  │  └─ (future)    │ │      │  (Brand configs)     │
                                 │  └─────────────────┘ │      └──────────────────────┘
                                 │                      │
                                 │  ┌─────────────────┐ │      ┌──────────────────────┐
                                 │  │ Shared Layer    │ │      │  Future:             │
                                 │  │  ├─ Auth        │ │ - - ►  Azure DevOps        │
                                 │  │  ├─ Validation  │ │      │  SonarQube           │
                                 │  │  ├─ Rate limit  │ │      │  Confluence          │
                                 │  │  └─ Logging     │ │      │  Nanobanana/Gemini   │
                                 │  └─────────────────┘ │      └──────────────────────┘
                                 └──────────────────────┘
```

Transport

- **stdio** (`StdioServerTransport`) — env var auth, per-developer (local development)
- **Streamable HTTP** (`StreamableHTTPServerTransport`) — production transport for hosted deployment
- **Auth:** per-user SSO via OAuth 2.1 — users authenticate with existing corporate credentials (e.g. Jira board access validated via SSO)

Runtime (Phase 1)

Stateless, local process launched per-session, single client per process, no caching layer.

3. Architecture Approach

MCP Server Pipeline

```
MCP Client Request
        │
        ▼
┌──────────────────┐
│ Transport Layer  │  (stdio / Streamable HTTP)
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ MCP Protocol     │  (JSON-RPC routing — SDK handles)
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Input Validation │  (Zod — automatic via SDK)
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ [Rate Limit]     │  (middleware slot — hooks only in Phase 1)
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Tool Handler     │  (validate → execute → format)
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Error Handling   │  (typed results, MCP error codes, logging)
└──────────────────┘
        │
        ▼
  MCP Client Response
```

- Input validation is automatic (SDK parses Zod schemas before handler runs)
- Output validation is manual (external API responses validated with Zod inside handlers)
- Error handling wraps everything — handlers return typed results; unhandled errors caught at transport layer

Tool Context

Handlers receive a lightweight `ToolContext` object providing access to the server and validated config:

```
export interface ToolContext {
  server: McpServer;
  config: ServerConfig;
}
```

Tool Handler Pattern

Functional with registration functions. Each tool domain is a directory with one file per operation. Adding a new operation = new file + one line in the domain's `index.ts`.

Error Hierarchy

```
ToolError (base)
├── ExternalServiceError    — API failures, timeouts
├── ValidationError         — Zod parse failure on API responses
└── ConfigurationError      — Missing env var, invalid config at startup
```

Client-facing errors are generic (OWASP). Structured logging captures detail server-side with correlation IDs via `console.error` (stdout reserved for MCP protocol in stdio transport).

Evolution Triggers

- **Tool count exceeds 10** — Consider auto-discovery registry or plugin system
- **Multiple tools share complex auth** — Extract auth into shared middleware

---

4. Project Structure

```
mcp-relay/
├── docs/
│   ├── architecture/
│   │   ├── architecture.md              # System design (this document)
│   │   └── supporting-docs/             # Detailed reference material
│   │       ├── key-decisions.md
│   │       ├── data-flow.md
│   │       ├── external-services.md
│   │       ├── tech-stack.md
│   │       └── open-items.md
│   └── roadmap.md                       # Phase roadmap (single source of truth)
│
├── src/
│   ├── server.ts                    # createServer(config) factory — registers all tools
│   ├── types.ts                     # ToolContext interface, shared types
│   │
│   ├── transports/
│   │   ├── stdio.ts                 # Entry point: stdio transport (Phase 1)
│   │   └── http.ts                  # Entry point: Streamable HTTP (Phase 1→2 gate)
│   │
│   ├── tools/
│   │   ├── jira/
│   │   │   ├── index.ts             # registerJiraTools(context)
│   │   │   ├── get-issue.ts         # jira_get_issue handler + input schema
│   │   │   ├── create-issue.ts      # jira_create_issue handler + input schema
│   │   │   ├── search-issues.ts     # jira_search_issues handler + input schema
│   │   │   ├── update-issue.ts      # jira_update_issue handler + input schema
│   │   │   └── schemas.ts           # Shared Jira Zod schemas (response types)
│   │   │
│   │   └── brand-guidelines/
│   │       ├── index.ts             # registerBrandTools(context)
│   │       ├── get-guidelines.ts    # brand_get_guidelines handler + input schema
│   │       └── schemas.ts           # Brand config Zod schemas
│   │
│   ├── shared/
│   │   ├── errors.ts                # ToolError hierarchy (ExternalServiceError, etc.)
│   │   ├── logger.ts                # Structured JSON logger (console.error for stdio)
│   │   ├── http-client.ts           # Thin fetch wrapper (timeouts, error wrapping)
│   │   └── rate-limit.ts            # Rate limit config shape + middleware slot
│   │
│   └── config/
│       ├── index.ts                 # Exports validated config object
│       └── schemas.ts               # Zod schemas for all env vars
│
├── .husky/
│   ├── pre-commit                   # pnpm lint && pnpm format:check
│   └── pre-push                     # pnpm lint && pnpm format:check && pnpm typecheck && pnpm build
│
├── .claude/                          # Claude Code configuration
│
├── .env.example                      # Template with all required env vars (no values)
├── .env                              # Actual credentials (gitignored)
├── .gitignore
├── .prettierrc                       # Prettier config
├── eslint.config.js                  # ESLint flat config (ESM)
├── package.json
├── tsconfig.json
└── README.md
```

Dependency Flow

```
transports/stdio.ts ──► server.ts ──► tools/jira/index.ts ──► tools/jira/get-issue.ts
transports/http.ts  ──►     │     ──► tools/brand/index.ts ──► tools/brand/get-guidelines.ts
                            │                   │
                            ▼                   ▼
                      types.ts           shared/errors.ts
                      config/index.ts    shared/logger.ts
                                         shared/http-client.ts
                                         config/index.ts
```

Rules:

- Transport entry points depend on `server.ts` and `config/` only — never on tools directly.
- `server.ts` depends on tool registration functions and `types.ts` — never on tool internals.
- Tool handlers receive `ToolContext` and depend on `shared/` and `config/` — never on each other.
- `shared/` has no dependencies on tools or config (standalone utilities).
- `config/` is a leaf — depends on nothing internal.
- No `src/index.ts` barrel file. The entry points are the transport files.

---

5. Visual References

- **System Context Diagram**: Miro widget
- **MCP Server Request Pipeline**: Miro widget
- **Jira Tool Data Flow**: Miro widget
- **Brand Guidelines Data Flow**: Miro widget
- **MCP Server Team Rollout Flow**: Miro widget
- **MCP Server Team AI Tooling Platform**: Miro widget
- **MCP Server Naming Convention**: Miro widget

---

6. Supporting Documentation

- **Key Decisions** — Resolved and deferred architecture questions with rationale
- **Data Flow** — Per-tool data flow diagrams and Zod schema inventories
- **External Services** — Jira, S3, and future service integration details
- **Tech Stack** — Full stack table, dev tooling, and future considerations
- **Open Items** — Risks, Phase 2+ MCP primitives, spikes, and phase roadmap